Enterprise AI

Contract Intelligence Platform

Technical Documentation

Azure OpenAI · FastAPI · Vector Search (RAG)

Author: Arnold Nemeth

Version: MVP v1.0

Status: Working MVP (AI core) — enterprise integrations designed

Date: July 2026

Table of Contents

1. Executive Summary

The Enterprise AI Contract Intelligence Platform is a demonstration of how modern generative AI can automate the first-level review of business contracts. Large organizations manage thousands of agreements, and legal teams review them manually — a slow, expensive, and error-prone process. This platform applies a Large Language Model (LLM) to read each contract, score its risk, produce an executive summary, and make the entire contract corpus queryable in natural language.

The solution is built on Azure OpenAI for both chat completions and text embeddings, a FastAPI REST backend, and a local ChromaDB vector store that provides semantic (meaning-based) search through Retrieval-Augmented Generation (RAG). The architecture deliberately mirrors the shape of enterprise AI solutions delivered by firms such as IBM, Microsoft, Deloitte, and Accenture, while remaining small enough to run on a single workstation.

This document distinguishes carefully between what is **verified working in the code** and what is **designed but not yet implemented**. The AI core — ingestion, embedding, risk analysis, semantic search, and RAG question answering — runs end to end. The enterprise integration layer (SharePoint document management and Power Automate orchestration) is fully designed and documented here but is not yet present in the repository.

At a glance

Dimension	Detail
Primary use case	Automated contract risk analysis and knowledge retrieval
AI provider	Azure OpenAI (chat: gpt-4o-mini; embeddings: embedding-test deployment)
Backend	Python 3.10+, FastAPI, Uvicorn
Vector store	ChromaDB (local, persistent), cosine similarity
Core pattern	Retrieval-Augmented Generation (RAG)
Interface	REST API + Swagger UI
Maturity	Working MVP for the AI core; ~40–45% toward production-ready

2. Business Problem and Use Case

In a large enterprise, contracts accumulate across procurement, sales, HR, IT, and facilities. At any given moment nobody has a reliable, consolidated view of where the risk lies. Specific pain points include:

- Unknown financial exposure — which contracts carry large values, penalties, or unfavorable payment terms.
- Compliance and GDPR risk — which agreements handle personal data or breach regulatory requirements.
- Liability — which contracts assign unlimited or one-sided liability.
- Payment terms — which vendors have unusually long terms (e.g., Net 120 or Net 180 days).
- Renewal and expiration tracking — which contracts end soon and require action.
- Vendor management — which vendors concentrate the most risk across their agreements.

Today this analysis is performed manually by legal and procurement staff. It does not scale: reviewing thousands of documents by hand is slow and inconsistent. The goal of this platform is to let AI perform the first-level review — surfacing risk, summarizing key terms, and answering questions — so that human experts can focus their attention where it matters most.

Representative questions the platform answers

```
Which contracts contain payment risks?
Which vendors have GDPR issues?
Show contracts with termination clauses.
List all high-risk contracts.
Which contracts expire this month?
Show contracts with unlimited liability.
Compare ACME and IBM contracts.
```

3. Solution Overview

The platform ingests contract text, converts it into a numerical vector (an embedding) that captures meaning, and stores it in a vector database alongside metadata. When a contract is uploaded, the system also sends it to the LLM for a structured risk assessment. When a user asks a question, the system embeds the question, retrieves the most semantically similar contracts, and passes them to the LLM as grounding context — the RAG pattern — so the answer is based on the organization's own documents rather than the model's general knowledge.

Why semantic search matters: traditional keyword search only finds exact matches. Embedding-based search understands meaning, so a query for "payment" also surfaces "invoice payment," "extended payment terms," "late payment," and "payment obligation" — without requiring the exact word to appear.

4. System Architecture

The diagram below shows the target end-state architecture. Components are annotated with their current status.

```
                |
                v
  Enterprise AI Answers / Risk JSON      [implemented]
                |
                v
   SharePoint Contract Register          [designed]
                |
                v
         Risk Dashboard                  [partial]
```

Design principle — modularity. Each component (LLM, vector store, ingestion, UI) can be replaced without affecting the others. The local ChromaDB store is intentionally simple to demonstrate the RAG architecture clearly, and is designed to be swapped for Azure AI Search in production.

5. Technology Stack

Each technology below plays a specific role. Where a technology is part of the designed-but-not-yet-implemented layer, it is marked accordingly.

Technology	Role	Status
Python 3.10+	Implementation language	Implemented
FastAPI	REST API framework serving all endpoints	Implemented
Uvicorn	ASGI server that runs the FastAPI app	Implemented
Azure OpenAI — Chat	Contract risk analysis and RAG answers (gpt-4o-mini)	Implemented
Azure OpenAI — Embeddings	Converts text to vectors for semantic search	Implemented
ChromaDB	Local persistent vector database (cosine similarity)	Implemented
Swagger UI	Interactive API documentation and testing	Implemented (built into FastAPI)
MCP (FastMCP)	Prototype tool server; future agent integration	Prototype only
SharePoint Online	Document repository + metadata register	Designed
Power Automate	Workflow automation / orchestration	Designed
Azure AI Search	Production-grade vector database	Planned
Azure AI Document Intelligence	OCR for PDF/Word/scanned contracts	Planned

Why Azure OpenAI instead of the public OpenAI API

Azure OpenAI provides the same models within an enterprise governance boundary: data residency, private networking, role-based access, and compliance certifications that enterprises require. Using the Azure variant is what makes this a credible enterprise design rather than a hobby integration.

6. Component Deep-Dive

6.1 FastAPI Backend — api/main.py

The FastAPI application is the system's entry point. On startup it loads environment variables from .env, constructs an AzureOpenAI client, and registers the REST endpoints. It imports two functions from the

embedding engine — add_document and search_similar — which encapsulate all vector-store interaction.

The backend has two core responsibilities: contract ingestion with risk analysis (/upload-contract) and knowledge retrieval (/ask, /semantic-search, /find-risky-contracts). It also exposes health checks and a dashboard endpoint.

6.2 Embedding Engine — vector_store/embedding_engine.py

This module wraps ChromaDB and the Azure embedding deployment. ChromaDB is initialized as a persistent client writing to ./vector_db, using a collection named enterprise_knowledge. Three functions form the interface:

- create_embedding(text) — calls the Azure embedding deployment and returns the vector.
- add_document(text, filename, vendor, risk_level) — embeds the text and stores it with metadata and a generated UUID.
- search_similar(query, top_k=3) — embeds the query, runs a cosine-similarity search, and returns results with a similarity score computed as (1 − distance).

The following is the search function, which shapes every result the API returns:

```
def search_similar(query, top_k=3):
    query_embedding = create_embedding(query)
    results = collection.query(
        query_embeddings=[query_embedding],
        n_results=top_k
    )
    formatted_results = []
    documents = results["documents"][0]
    metadatas = results["metadatas"][0]
    distances = results["distances"][0]
    for i in range(len(documents)):
        formatted_results.append({
            "text": documents[i],
            "filename": metadatas[i]["filename"],
            "vendor": metadatas[i]["vendor"],
            "risk_level": metadatas[i]["risk_level"],
            "score": round(1 - distances[i], 3)
        })
    return formatted_results
```

6.3 MCP Server (Prototype) — mcp_server/server.py

A minimal FastMCP server exposes two tools — search_documents (keyword search over local files) and invoice_lookup (a stub returning a hardcoded invoice status). This is a leftover from an earlier direction and is not part of the production workflow. It is retained as a foundation for future integration, for example exposing the platform's capabilities to an MCP-compatible client or AI agent.

6.4 Sample Data — documents/

The repository ships several short sample contracts used to populate and test the vector store. They intentionally contain risk signals — long payment terms, immediate termination, broad liability, and SLA penalties — so the risk analysis has something meaningful to detect. Example:

```
Enterprise Service Agreement
Client: Global Retail AG
Vendor: SmartVision AI
Contract Value: 1,200,000 EUR
Payment Terms: Net 120 days.
Termination: Client may terminate immediately without compensation.
Security: Vendor liable for all data breaches.
Penalties: 10% penalties for SLA violations.
```

7. API Reference

The API exposes seven application endpoints plus the auto-generated Swagger UI at /docs. Endpoints that call Azure OpenAI require a valid key and network access.

Method	Path	Purpose	Azure
GET	/	Health check	No
POST	/health-test	Status probe	No
GET	/dashboard	Contract counts by risk level	No*
POST	/upload-contract	Embed contract + return risk JSON	Yes
POST	/ask	RAG: search then LLM answer	Yes
POST	/semantic-search	Return most similar contracts	Yes
POST	/find-risky-contracts	Filter results for risk keywords	Yes

* /dashboard does not call Azure, but currently returns zeros due to a known bug (see Section 9).

POST /upload-contract

Accepts a multipart file upload. Reads and UTF-8 decodes the contract, stores it in the vector database, then sends it to the LLM for structured risk analysis. Returns strict JSON.

Example response:

```
{
  "risk_score": 85,
  "risk_level": "High",
  "financial_risk": "Net 120 payment terms strain cash flow; 10% SLA penalties.",
  "compliance_risk": "No explicit GDPR data-processing terms.",
  "liability_risk": "Vendor liable for all data breaches (broad, uncapped).",
  "operational_risk": "Immediate termination without compensation.",
  "executive_summary": "High-value agreement with aggressive terms..."
}
```

POST /ask

The RAG endpoint. It accepts a JSON body { "question": "..." }, runs a semantic search, assembles an enterprise context block (filename, vendor, risk level, similarity score, and text for each retrieved document), and sends that context plus the question to the LLM. The response contains the answer, the cited semantic results, and a result count.

```
Request:   { "question": "Which vendors have long payment terms?" }
Response:  {
  "question": "...",
  "answer": "SmartVision AI appears in multiple contracts with Net 120...",
  "semantic_results": [ { "filename": "...", "vendor": "...", "score": 0.83, ... } ],
  "results_count": 3
}
```

POST /semantic-search

Returns the raw most-similar contracts for a query, without an LLM call on top. Useful for debugging retrieval quality.

POST /find-risky-contracts

Runs a semantic search and then filters the results, keeping any whose text contains risk-signal keywords: risk, penalty, termination, compliance, gdpr, liability, lawsuit, or breach. Returns the matching contracts and a count.

8. Vector Search and RAG Design

Every stored contract is converted to an embedding — a high-dimensional numeric vector that encodes semantic meaning. Similarity between a query and stored documents is measured by cosine similarity, and ChromaDB returns the nearest neighbors. The platform reports a similarity score of (1 − distance) so that higher is better.

Retrieval-Augmented Generation combines this retrieval step with generation. Rather than asking the LLM to answer from its own training, the system first retrieves the organization's most relevant contracts and injects them into the prompt as authoritative context. This grounds answers in real documents, reduces hallucination, and lets the model cite specific filenames. The same pattern underpins Microsoft Copilot, IBM watsonx Assistant, and ChatGPT Enterprise's document features.

Stored per document

- embedding — the vector representation
- text — the original contract content
- filename — source file name
- vendor — vendor / counterparty
- risk_level — risk classification

9. Prompt Engineering

The platform uses structured, enterprise-oriented prompts rather than generic "summarize this" instructions. The risk-analysis prompt directs the model to identify specific categories of risk and to return strict JSON so downstream systems can consume the output directly. The system message casts the model as a senior enterprise contract risk analyst and forbids markdown or prose around the JSON.

Risk-analysis prompt (abridged):

```
Analyze this enterprise contract. Identify:
- overall risk score (0-100)
- risk level (Low / Medium / High)
- financial risks
- compliance concerns
- GDPR risks
- liability risks
- operational risks
- suspicious clauses
```

```
Return ONLY valid JSON in this exact format:
{ "risk_score": 75, "risk_level": "High", "financial_risk": "...",
  "compliance_risk": "...", "liability_risk": "...",
  "operational_risk": "...", "executive_summary": "..." }
```

The RAG prompt (used by /ask) instructs the model to answer professionally, use the supplied enterprise knowledge, mention filenames where relevant, and flag GDPR / compliance / liability concerns. Temperature is kept low (0.2–0.3) to favor consistent, factual output over creativity.

10. Azure Configuration

The platform requires an Azure OpenAI resource with two deployments: a chat model and an embedding model. Configuration is supplied entirely through environment variables, so no credentials are hardcoded.

Variable	Purpose
AZURE_OPENAI_API_KEY	Secret key for the Azure OpenAI resource
AZURE_OPENAI_ENDPOINT	Resource endpoint URL (https://<name>.openai.azure.com/)
AZURE_OPENAI_API_VERSION	API version (e.g. 2024-10-21)
AZURE_OPENAI_DEPLOYMENT	Chat deployment name (e.g. gpt-4o-mini)
AZURE_OPENAI_EMBEDDING_DEPLOYMENT	Embedding deployment name (e.g. embedding-test)

11. SharePoint Architecture (Designed)

The SharePoint layer is designed but not yet implemented in code. It separates physical files from searchable metadata using two objects.

Contracts Repository

A document library that stores the physical files (contract.pdf, agreement.docx, nda.pdf, and so on). It holds documents, not metadata.

Contract Register

A SharePoint list that stores searchable metadata about each contract — this is what management works with day to day. Planned columns:

Column	Description
Title	Contract title
Vendor	Counterparty / supplier
RiskScore	0–100 numeric score from the AI
RiskLevel	Low / Medium / High
ContractStatus	Workflow status
ExecutiveSummary	AI-generated summary
UploadDate	When ingested
ReviewedBy / ReviewDate	Human review audit trail
FileLink	Link to the file in the Repository

12. Power Automate Flow (Designed)

Power Automate is intended to orchestrate the end-to-end flow, connecting SharePoint to the FastAPI backend and writing results back automatically. The current (simple) flow and the planned (complete) flow are shown below.

Current (simple)

```
When file created -> Get file content -> Create SharePoint item
```

Planned (complete)

```
When file uploaded
  -> Get file content
  -> HTTP POST to FastAPI /upload-contract
  -> Receive AI JSON (risk_score, risk_level, summary...)
  -> Update SharePoint Contract Register
  -> Send Teams notification
```

13. Installation and Setup Guide

The application runs on Python 3.10 or later. Run all commands from the project root, because the vector-store path is resolved relative to the working directory.

```
# 1. Create and activate a virtual environment
python -m venv venv
venv\Scripts\activate          # Windows
# source venv/bin/activate       # macOS / Linux

# 2. Install dependencies
pip install -r requirements.txt

# 3. Configure Azure OpenAI
copy .env.example .env          # then fill in your values

# 4. Run the API (from the project root)
uvicorn api.main:app --reload

# 5. Open the interactive docs
#    http://127.0.0.1:8000/docs
```

Environment note: ChromaDB's SQLite backend can fail with a "disk I/O error" on some network or FUSE-mounted filesystems. Run the application from a local disk. This is an environment limitation, not a defect in the application.

14. Configuration

All configuration is via the .env file, which is never committed. A .env.example template ships in the repository with variable names only. The pinned, verified runtime dependencies are captured in requirements.txt: FastAPI, Uvicorn, python-multipart (required for file uploads), the OpenAI SDK (AzureOpenAI client), ChromaDB, python-dotenv, and the MCP package for the prototype server.

15. Security Considerations

Secret management is the most important immediate concern. A live Azure OpenAI API key was present in the .env file in the working tree. Because .env sits in the folder that would be pushed to source control, that key would leak the moment the repository is published. Recommended actions:

- Rotate the exposed key immediately in the Azure Portal.
- Never commit .env — the shipped .gitignore excludes it, and .env.example documents the required variables without secrets.
- For production, source secrets from Azure Key Vault or Managed Identity rather than a file.

Beyond secrets, a production deployment should add authentication and authorization (Azure Entra ID), transport security, request validation, rate limiting, and audit logging. These are tracked in the roadmap.

16. Current Status: Verified vs. Planned

The table below reflects what is actually present and running in the repository. An item is marked Done only if it exists and runs in code.

Item	Status	Notes
FastAPI backend + Swagger	Done	Boots; 7 app endpoints
Azure OpenAI chat + embeddings	Done	gpt-4o-mini + embedding deployment
ChromaDB vector store + search	Done	Persistent; cosine similarity
RAG /ask	Done	Search -> context -> LLM
Risk analysis /upload-contract	Done	Fixed: stores real AI-derived vendor/risk
/dashboard	Done	Fixed: queries ChromaDB for real counts
requirements.txt / .env.example / .gitignore	Done	Added in this pass
MCP server	Prototype	Not in production workflow
SharePoint integration	Designed	No code yet
Power Automate flow	Designed	No code yet
Auth / Azure deploy / OCR / Azure AI Search	Planned	Future phases

Completion estimate

Layer	Estimate
Core AI pipeline (ingest -> embed -> analyze -> RAG)	~90%
MVP end-to-end (incl. SharePoint / Power Automate loop)	~60%
Production-ready enterprise solution	~40–45%

17. Known Issues and Recommended Fixes

17.1 /dashboard returned zeros — FIXED

The endpoint read vector_store/vector_db.json, a file that does not exist because the project moved to ChromaDB, so it always reported zero contracts. It now calls a new get_collection_stats() helper that

queries the Chroma collection directly and aggregates counts by risk level and by vendor. Verified: it returns the true document count instead of zeros.

17.2 /upload-contract stored hardcoded metadata — FIXED

The endpoint previously passed vendor="ACME" and risk_level="High" literally, so every stored document was labeled High regardless of the AI's analysis. It now runs the risk analysis first (the prompt also extracts the vendor), parses the JSON — tolerating stray code fences — and stores the real vendor and risk_level. If the model ever returns invalid JSON, the contract is still stored with Unknown metadata so it remains searchable.

17.3 Stale metadata on existing embeddings

The documents already in the vector store were ingested by the old code and still carry vendor=ACME / risk_level=High. A reindex.py script re-embeds the files in documents/ with correct AI-derived metadata; run it (optionally with --reset) after the fixes to make the store accurate.

17.4 Repository hygiene

Legacy FAISS artifacts (faiss.index, documents.pkl) remain alongside the active ChromaDB store, and several "- Copy" duplicate files and double-extension sample files (*.txt.txt) clutter the tree. These are safe to remove and are now excluded by .gitignore where applicable.

18. Roadmap

The near-term priority is to finish the existing end-to-end scenario rather than adding new surface area — a complete, demonstrable loop is more valuable than a broader but half-finished feature set.

Phase 0 — Stabilize

- Rotate the exposed key; keep .env out of source control.
- Fix /dashboard to query ChromaDB.
- Fix /upload-contract metadata storage.
- Remove legacy artifacts and duplicates.

Phase 1 — Complete the MVP loop

- Power Automate -> HTTP -> FastAPI integration.
- Automatic write-back to the SharePoint Contract Register.
- Working dashboard backed by real data.
- Vendor analytics and multi-contract questions.

Phase 2 — Production hardening

- Authentication (Azure Entra ID).
- Replace ChromaDB with Azure AI Search.
- OCR via Azure AI Document Intelligence.
- Logging, monitoring, CI/CD, Azure deployment, Power BI dashboard.

Phase 3 — Advanced intelligence

- Expiration alerts and renewal reminders.
- Clause extraction and comparison.
- Contract versioning and approval workflows.
- AI recommendations.

Phase 4 — Multi-agent platform

- Contract, Legal, Compliance, Finance, and Procurement agents.
- Manager Copilot for natural-language governance.
- Copilot Studio integration.

19. Business Value and Skills Demonstrated

For an organization, the platform reduces the time and cost of first-level contract review, improves consistency, and surfaces risk that manual review might miss. For a portfolio or technical interview, it demonstrates an end-to-end enterprise AI architecture across a broad set of skills:

- Generative AI and Azure OpenAI
- Prompt engineering with structured JSON output
- REST API design with FastAPI
- Embeddings, vector databases, and semantic search
- Retrieval-Augmented Generation (RAG)
- Enterprise architecture and solution design
- Microsoft Power Platform and SharePoint integration (designed)
- Risk analysis and contract analytics
- Knowledge management and AI-assisted governance

20. Lessons Learned

Documentation should track reality, not intention. The original README described an aspirational structure — a FAISS/JSON vector store and a requirements.txt that were not actually in the repository. Reconciling the documentation against the source revealed genuine defects (the dashboard bug, the hardcoded upload metadata) and a missing dependency file. The lesson: keep a clear, honest separation between what is built and what is planned, and verify claims against the code.

Structured LLM output pays off. Forcing the risk analysis into strict JSON makes the model's output directly consumable by SharePoint, dashboards, and automation — turning a chat feature into a data pipeline.

Keep the architecture modular. Because retrieval, generation, and storage are decoupled, the simple local vector store can be replaced with Azure AI Search later without rewriting the API or the ingestion logic. That replaceability is what allows an MVP to grow into a production system incrementally.

21. End-to-End Worked Example

This walkthrough traces a single contract through the platform, from upload to a natural-language query, using one of the bundled sample documents.

Step 1 — Upload a contract

A user (or Power Automate, in the designed flow) sends the contract to /upload-contract as a multipart file. The backend decodes the text, embeds it, stores it in ChromaDB, and asks the LLM to analyze it.

```
Input file: high_risk_contract.txt
  "Vendor payment terms are 180 days.
   Client may terminate immediately without notice.
   Vendor assumes liability for all data breaches."
```

Step 2 — Receive structured risk analysis

The model returns strict JSON. Downstream systems can store these fields directly in the SharePoint Contract Register without any parsing of prose.

```
{
  "risk_score": 88,
  "risk_level": "High",
  "financial_risk": "180-day payment terms severely delay vendor cash flow.",
  "compliance_risk": "No GDPR / data-processing provisions present.",
  "liability_risk": "Vendor assumes liability for ALL data breaches (uncapped).",
  "operational_risk": "Client may terminate immediately without notice.",
  "executive_summary": "A one-sided, high-risk agreement: extended payment
   terms, immediate termination rights for the client, and unlimited vendor
   liability. Recommend legal review before signature."
}
```

Step 3 — Ask a natural-language question

Later, a manager asks a question. The platform embeds the question, retrieves the most similar contracts (including the one above), and grounds the LLM's answer in them.

```
POST /ask
{ "question": "Which contracts have the worst payment terms?" }


-> semantic search retrieves contracts mentioning Net 120 / 180 days
-> LLM answers, citing filenames and flagging the liability clause
-> response includes answer + semantic_results + results_count
```

The key point: the answer is grounded in the organization's own contracts, not the model's general knowledge, and every claim can be traced back to a source file.

22. Testing and Verification

The application was set up in a clean environment and the baseline verified before any changes were made. The findings below establish a known-good starting point.

Environment

A fresh Python 3.10 virtual environment was created and the runtime dependencies installed from the reconstructed requirements.txt (FastAPI, Uvicorn, the OpenAI SDK, ChromaDB, python-dotenv, python-multipart, MCP). The application imported cleanly and ChromaDB initialized, loading an existing collection of sample-contract embeddings.

Boot and route check

Importing the FastAPI app succeeded and registered all expected routes: /, /ask, /upload-contract, /semantic-search, /find-risky-contracts, /dashboard, /health-test, plus Swagger UI at /docs and the OpenAPI schema at /openapi.json.

Offline endpoint results

Endpoint	Result	Observation
GET /	200 OK	{"message":"Enterprise AI API is running"}
POST /health-test	200 OK	{"status":"working"}
GET /dashboard	200 OK	Pre-fix: returned zeros despite a populated store (confirmed the bug)

Post-fix verification

After the fixes described in Section 17, /dashboard was re-tested against the same vector store and now returns the true document count (8) with a per-vendor breakdown, instead of zeros. The /upload-contract fix was code-reviewed and its module imports cleanly; a full live test requires a valid Azure key and should be run once the exposed key is rotated.

Azure-dependent endpoints (/ask, /upload-contract, /semantic-search, /find-risky-contracts) were not exercised against the live key, since the recommended action is to rotate the exposed key first. Once a fresh key is in place, a full end-to-end test of the RAG pipeline can be run.

Recommended test matrix (next)

- Upload each sample contract and assert the risk JSON parses and risk_level is plausible.
- Query /ask with known questions and assert the expected filenames appear in semantic_results.
- Confirm /dashboard reflects true counts after the fix.
- Add automated tests (pytest + FastAPI TestClient) so regressions are caught.

23. Deployment Options

For production, the FastAPI application can be hosted on any of several Azure compute services. The choice trades operational simplicity against control and scaling flexibility.

Option	Best for	Trade-offs
Azure App Service	Simple, always-on web APIs	Easiest path; managed platform; less control over the runtime
Azure Functions	Event-driven / bursty workloads	Scales to zero; cost-efficient for spiky traffic; cold starts
Azure Container Apps	Containerized microservices	More control and portability; requires container tooling

Whichever host is chosen, the vector store should move from local ChromaDB to Azure AI Search, and secrets should come from Azure Key Vault or a Managed Identity rather than a file. A CI/CD pipeline (GitHub Actions or Azure DevOps) should build, test, and deploy on each change.

24. Cost and Scaling Considerations

Operating cost is driven mainly by Azure OpenAI token usage. Two levers dominate:

- **Embeddings** — charged per token embedded. Each document is embedded once on ingestion, and each query is embedded once. This is inexpensive relative to chat.
- **Chat completions** — charged per input and output token. The RAG prompt includes retrieved contract text, so prompt size grows with the amount of context supplied. Keeping top_k modest (3 by default) controls cost.

Scaling considerations as the corpus grows: local ChromaDB is fine for hundreds to a few thousand documents on one machine, but a production corpus of tens of thousands of contracts should use Azure AI Search for managed scaling, filtering, and hybrid (keyword + vector) retrieval. Chunking long contracts into sections before embedding improves retrieval precision and keeps individual prompts small. Caching frequent queries and their answers further reduces token spend.

25. Comparison with Commercial Enterprise AI Platforms

The architecture here is a scaled-down version of the same pattern used by major commercial products. Understanding the parallels clarifies both what the platform demonstrates and where it deliberately stops short.

Capability	This platform	Commercial (Copilot / watsonx / ChatGPT Enterprise)
RAG over private documents	Yes (ChromaDB + Azure OpenAI)	Yes, at enterprise scale
Structured risk extraction	Yes (strict JSON prompts)	Varies; often via custom skills
Vector database	Local ChromaDB	Managed (Azure AI Search, Elastic, etc.)
Authentication / RBAC	Not yet	Entra ID / SSO built in
Document OCR	Planned	Built in (Document Intelligence)
Agents / orchestration	Prototype (MCP)	Copilot Studio / watsonx Assistant
Governance / audit	Designed	Enterprise-grade

The platform's value is that it demonstrates the full pattern — retrieval, grounding, structured output, and an enterprise integration design — end to end and transparently, using components that map directly onto their production-grade equivalents.

26. Data Model and Metadata Schema

Each contract is stored in ChromaDB as an entry combining the embedding vector, the raw text, a unique identifier, and a small metadata record. This metadata is what makes filtered and faceted retrieval possible, and it maps directly onto the columns of the designed SharePoint Contract Register.

Field	Type	Source	Purpose
id	UUID (string)	Generated on ingest	Unique document key
embedding	float vector	Azure embeddings	Semantic similarity search
text	string	Uploaded file	Original contract content / RAG context

Field	Type	Source	Purpose
filename	string	Upload metadata	Traceability / citation
vendor	string	AI analysis (planned)	Vendor analytics and filtering
risk_level	string	AI analysis (planned)	Low / Medium / High classification

Note: today the vendor and risk_level fields are populated with hardcoded placeholders at ingest (see Section 17.2). The intended design — and the fix — is to populate them from the model's risk analysis so that stored metadata reflects reality and downstream analytics are trustworthy.

27. Error Handling and Edge Cases

The current implementation handles the most important failure modes and leaves others for hardening. Documented behavior:

- **Invalid model JSON.** If the LLM returns text that is not valid JSON, /upload-contract catches the parse error and returns a structured error object containing the raw response, rather than crashing.
- **Empty retrieval.** If semantic search finds nothing relevant, /ask substitutes a clear "No relevant enterprise knowledge found" context so the model does not fabricate sources.
- **Mixed result types.** The /ask handler defensively supports dict, string, and unknown result shapes when building context, avoiding attribute errors.

Edge cases to address during hardening: non-UTF-8 or binary uploads (currently assumes UTF-8 text — OCR/decoding needed for PDFs); very large contracts that exceed the model's context window (needs chunking); Azure rate limits and transient network errors (needs retries with backoff); and concurrent writes to the vector store.

28. Maintenance and Operations

Day-to-day operation of the platform involves a small number of recurring tasks:

- Dependency updates — periodically refresh pinned versions in requirements.txt and re-run the test matrix.
- Key rotation — rotate the Azure OpenAI key on a schedule and immediately if exposure is suspected.
- Vector-store maintenance — the local ChromaDB store lives in ./vector_db; back it up before migrations, and rebuild embeddings if the embedding model changes.
- Monitoring — in production, track request latency, error rates, and Azure token consumption; alert on anomalies.
- Repository hygiene — keep legacy artifacts (FAISS files, '- Copy' duplicates) out of source control via .gitignore.

Because embeddings are tied to the model that produced them, changing the embedding deployment requires re-embedding the entire corpus. Plan such migrations as a batch job and validate retrieval quality afterward.

29. Frequently Asked Questions

Why ChromaDB instead of Azure AI Search?

ChromaDB keeps the MVP simple and fully local, which makes the RAG architecture easy to understand and demonstrate. The design treats the vector store as replaceable, so Azure AI Search can be introduced for production scale without rewriting the API.

Does the platform send my contracts to the public OpenAI?

No. It uses Azure OpenAI, which runs within the enterprise's Azure tenant and governance boundary. Data stays within the Azure environment configured by the organization.

Can it read PDFs and scanned documents?

Not yet. The current pipeline assumes UTF-8 text. OCR via Azure AI Document Intelligence is on the roadmap to support PDF, Word, and scanned contracts.

How accurate is the AI risk analysis?

It is a first-level triage aid, not a substitute for legal review. Low temperature and structured prompts make output consistent, but a human should review high-stakes contracts. The audit-trail fields (ReviewedBy, ReviewDate) exist precisely to record that human sign-off.

Is the MCP server required to run the platform?

No. The FastAPI backend is the production surface. The MCP server is a separate prototype retained for future agent integration and is not part of the main workflow.

Appendix A. Glossary

Term	Meaning
Embedding	A numeric vector representing the meaning of text
Vector store	A database optimized for similarity search over embeddings
Cosine similarity	A measure of how close two vectors point in the same direction
RAG	Retrieval-Augmented Generation — retrieve relevant docs, then generate
LLM	Large Language Model (here, gpt-4o-mini via Azure OpenAI)
MCP	Model Context Protocol — a standard for exposing tools to AI clients
Swagger UI	Interactive, auto-generated API documentation

Appendix B. File Inventory

Path	Role
api/main.py	FastAPI app and all REST endpoints
vector_store/embedding_engine.py	ChromaDB + Azure embeddings
vector_db/	ChromaDB persistent store (chroma.sqlite3)
documents/	Sample contracts
mcp_server/server.py	Prototype MCP server (not in production)
reindex.py	Re-embeds documents/ with correct AI-derived metadata

Path	Role
requirements.txt	Pinned runtime dependencies
.env / .env.example	Azure config (secret / template)
README, PROJECT_STATUS, ARCHITECTURE, ROADMAP, CHANGELOG	Repository documentation set